Taller 2
Simulación Montecarlo de un sistema de inventarios

Curso	Métodos Cuantitativos
Taller	Taller N.º 2
Fecha de entrega	Miércoles 28 de octubre de 2026
Modalidad	Individual
Herramientas	Microsoft Excel y Python

1. Objetivos y descripción del sistema

En este taller usted simulará un año completo de operación de un sistema de inventarios con una política de compra que usted mismo diseña. Hará 100 réplicas del año, primero en Excel y luego en Python, para dos distribuciones de la demanda.

Objetivos de aprendizaje

- Construir un modelo de simulación Montecarlo de un sistema de inventarios discreto.
- Generar variables aleatorias discretas (uniforme y empírica) con el método de la transformada inversa.
- Estimar medidas de desempeño (ganancia, ventas perdidas) a partir de 100 réplicas.
- Comparar políticas de compra y distribuciones de demanda con base en los resultados simulados.
- Implementar el mismo modelo en dos herramientas y contrastar sus resultados.

El sistema

Una empresa vende un único producto. Abre de lunes a sábado; el domingo no abre. Cada día llega una demanda aleatoria de unidades. Si hay inventario, se vende; las unidades que no se pueden atender se pierden (el cliente no espera). Al cierre de cada día se revisa el inventario y, según la política de compra, se decide si se hace un pedido al proveedor.

Parámetros económicos (pesos colombianos)

Concepto	Valor	Cómo se aplica
Precio de venta	\$680.000 por unidad	Por cada unidad vendida
Costo de compra	\$600.000 por unidad	Por cada unidad recibida
Costo por pedido	\$40.000 por pedido	Cada vez que se recibe un pedido
Seguros, almacenamiento y otros	\$80 por unidad por día	Sobre el inventario promedio del día hábil
Costos fijos	\$1.000.000 por mes	El primer día hábil de cada mes

Demanda diaria (unidades)

Se analizan dos casos. En ambos la demanda toma valores enteros entre 3 y 7.

Demanda	Caso 1: uniforme discreta	Caso 2: empírica
3	0,20	0,20
4	0,20	0,05
5	0,20	0,40
6	0,20	0,25
7	0,20	0,10

Política de compra (la decide usted)

Se usa una política (s, Q). Al cierre de cada día, si el inventario final es *estrictamente menor* que s (punto de reorden), se piden Q unidades. Q es constante durante todo el año. Usted elige los valores de s y Q, prueba varias combinaciones y justifica cuál recomienda.

2. Supuestos y lógica del modelo

El modelo simula día por día los 313 días hábiles de 2026 y acumula la ganancia del año. Ambas implementaciones (Excel y Python) deben respetar exactamente estos supuestos.

Supuestos

1. Horizonte: año 2026, de lunes a sábado, sin festivos (313 días hábiles). El domingo no hay demanda ni costo de almacenamiento.
2. Las demandas diarias son independientes entre sí y siguen la misma distribución todos los días.
3. El inventario se revisa al final de cada día. Si el inventario final es menor que s, se piden Q unidades.
4. El tiempo de entrega es cero: lo pedido al cierre de un día está disponible al inicio del siguiente día hábil (lo pedido el sábado llega el lunes).

5. Q es constante. El inventario inicial del primer día es Q y cuenta como un pedido recibido ese día (se paga el costo de compra y el costo por pedido).
6. La demanda no atendida es venta perdida; no hay pedidos pendientes de clientes.
7. Cada pedido se paga cuando se recibe. Un pedido hecho el último día del año llega en 2027 y no se cuenta.
8. El costo de almacenamiento de cada día se calcula sobre el inventario promedio del día: (inventario inicial + inventario final)/2.
9. Los costos fijos de $1.000.000 se cargan completos el primer día hábil de cada mes (12 cargos en el año).
10. El inventario que sobra al final del año no tiene valor residual.
11. No hay capacidad máxima de almacenamiento.

Secuencia de eventos de cada día hábil t

1. Se recibe el pedido hecho al cierre del día anterior, si lo hubo (el día 1 se reciben Q unidades).
2. Se calcula el inventario inicial.
3. Llega la demanda del día.
4. Se vende lo que alcance el inventario; el resto es venta perdida.
5. Se calcula el inventario final.
6. Se revisa el inventario final y se decide si se pide.
7. Se registran ingresos y costos del día.

Notación

Símbolo	Significado
R_t	Unidades recibidas al inicio del día t
II_t	Inventario inicial del día t
D_t	Demanda del día t
V_t	Unidades vendidas el día t
P_t	Ventas perdidas el día t
IF_t	Inventario final del día t
O_t	1 si se pide al cierre del día t, 0 si no
M_t	1 si t es el primer día hábil de un mes, 0 si no
U_t	Utilidad del día t
G	Ganancia del año (una réplica)

Ecuaciones del día

$$R_1 = Q, \qquad R_t = Q\,O_{t-1} \quad (t \geq 2) \tag{1}$$

$$II_1 = R_1, \qquad II_t = IF_{t-1} + R_t \quad (t \geq 2) \tag{2}$$

$$V_t = \min(II_t, D_t), \qquad P_t = D_t - V_t, \qquad IF_t = II_t - V_t \tag{3}$$

$$O_t = \begin{cases} 1 & \text{si } IF_t < s \\ 0 & \text{si no} \end{cases} \tag{4}$$

$$U_t = 680\,000\,V_t - 600\,000\,R_t - 40\,000\,O_{t-1} - 80\,\frac{II_t + IF_t}{2} - 1\,000\,000\,M_t \tag{5}$$

$$G = \sum_{t=1}^{313} U_t \tag{6}$$

Estadísticos sobre las 100 réplicas ($r = 1, \ldots, 100$; $n = 313$ días)

$$\bar{G} = \frac{1}{100}\sum_{r=1}^{100} G_r, \qquad S_G = \sqrt{\frac{1}{99}\sum_{r=1}^{100}\left(G_r - \bar{G}\right)^2} \tag{7}$$

$$\hat{p}_{\text{perdida}} = \frac{1}{100\,n}\sum_{r=1}^{100}\sum_{t=1}^{n} \mathbf{1}[P_{r,t} > 0], \qquad \bar{P} = \frac{1}{100\,n}\sum_{r=1}^{100}\sum_{t=1}^{n} P_{r,t} \tag{8}$$

La probabilidad de ventas perdidas es la fracción de días con al menos una unidad perdida. El promedio de ventas perdidas se mide en unidades por día.

Ejemplo para verificar su modelo

Con $s = 6$, $Q = 10$ y demandas fijas de 4, 7 y 5 unidades los tres primeros días, su modelo debe dar exactamente estos valores:

Día	Fecha	R	II	D	V	P	IF	Pedido	Utilidad del día
1	jue 01/01/2026	10	10	4	4	0	6	0	−$4.320.640
2	vie 02/01/2026	0	6	7	6	1	0	1	$4.079.760
3	sáb 03/01/2026	10	10	5	5	0	5	1	−$2.640.600

El día 1 incluye el costo fijo de enero y la compra inicial. El día 3 se recibe el pedido hecho el día 2; el pedido del día 3 llega el lunes 05/01/2026.

3. Parte A: modelo en Excel (50 %)

El libro tendrá tres hojas: `Parametros`, `Uniforme` y `Empirica`. Cada hoja de simulación contiene un año completo (una fila por día hábil) y una tabla de datos que lo repite 100 veces. Las fórmulas están escritas para Excel en español, con punto y coma (;) como separador de argumentos.

3.1. Paso A1. Hoja de parámetros

Cree la hoja `Parametros`. Escriba cada parámetro en su propia celda y asígnele un nombre (pestaña Fórmulas > Asignar nombre). Así las fórmulas se leen solas y, al cambiar un valor, todo el modelo se actualiza.

Celda	Parámetro	Valor	Nombre definido
B4	Punto de reorden s	(su elección)	`PuntoReorden`
B5	Cantidad a pedir Q	(su elección)	`CantPedido`
B6	Precio de venta	680000	`PrecioVenta`
B7	Costo de compra por unidad	600000	`CostoUnitario`
B8	Costo por pedido	40000	`CostoPedido`
B9	Costo de almacenamiento por unidad-día	80	`CostoAlm`
B10	Costo fijo mensual	1000000	`CostoFijo`
B11	Año simulado	2026	`AnioSim`

3.2. Paso A2. Tabla de la distribución empírica

En la misma hoja, construya la tabla para generar la demanda empírica con la transformada inversa. La primera columna es el límite inferior de la probabilidad acumulada: si un número aleatorio u cae en [0; 0,20) la demanda es 3, si cae en [0,20; 0,25) es 4, y así sucesivamente.

Fila	Lím. inf. acumulado (E)	Demanda (F)	Probabilidad (G)	Acumulada (H)
4	`0`	3	0,20	`=G4`
5	`=H4`	4	0,05	`=H4+G5`
6	`=H5`	5	0,40	`=H5+G6`
7	`=H6`	6	0,25	`=H6+G7`
8	`=H7`	7	0,10	`=H7+G8`

Asigne el nombre `TablaEmpirica` al rango `E4:F8`. Compruebe que la suma de `G4:G8` es 1.

3.3. Paso A3. Hoja de simulación de un año (demanda uniforme)

Cree la hoja `Uniforme`. Escriba los encabezados en la fila 5. La fila 6 es el día 1 y tiene fórmulas propias; la fila 7 tiene la fórmula general, que se copia hasta la fila 318 (313 días).

Col.	Encabezado	Fila 6 (día 1)	Fila 7 (copiar hasta la 318)
A	Día	`1`	`=A6+1`
B	Fecha	`=FECHA(AnioSim;1;1)`	`=B6+SI(DIASEM(B6;2)=6;2;1)`
C	Mes	`=MES(B6)`	`=MES(B7)`
D	Inicio de mes (1/0)	`1`	`=SI(C7<>C6;1;0)`
E	Unidades recibidas	`=CantPedido`	`=SI(L6=1;CantPedido;0)`
F	Inventario inicial	`=E6`	`=J6+E7`
G	Demanda	`=ALEATORIO.ENTRE(3;7)`	`=ALEATORIO.ENTRE(3;7)`
H	Ventas	`=MIN(F6;G6)`	`=MIN(F7;G7)`
I	Ventas perdidas	`=G6-H6`	`=G7-H7`
J	Inventario final	`=F6-H6`	`=F7-H7`
K	Día con faltante (1/0)	`=SI(I6>0;1;0)`	`=SI(I7>0;1;0)`
L	Pedido al cierre (1/0)	`=SI(J6<PuntoReorden;1;0)`	`=SI(J7<PuntoReorden;1;0)`
M	Ingresos	`=PrecioVenta*H6`	`=PrecioVenta*H7`
N	Costo de compra	`=CostoUnitario*E6`	`=CostoUnitario*E7`
O	Costo de pedido	`=SI(E6>0;CostoPedido;0)`	`=SI(E7>0;CostoPedido;0)`
P	Costo de almacenamiento	`=CostoAlm*(F6+J6)/2`	`=CostoAlm*(F7+J7)/2`
Q	Costo fijo	`=CostoFijo*D6`	`=CostoFijo*D7`
R	Utilidad del día	`=M6-N6-O6-P6-Q6`	`=M7-N7-O7-P7-Q7`

Por qué se construye así:

- **Fecha (B):** `DIASEM(B6;2)` devuelve 1 para lunes y 6 para sábado. Si el día es sábado, se suman 2 para saltar el domingo. El 1 de enero de 2026 es jueves, así que la primera fila ya es un día hábil.
- **Inicio de mes (D):** vale 1 cuando el mes cambia respecto a la fila anterior. Activa el costo fijo mensual.
- **Recepciones (E):** la decisión de pedir se toma al cierre del día anterior (columna L de la fila de arriba). Como el tiempo de entrega es cero, el pedido entra al inventario inicial del día siguiente.
- **Ventas (H):** no se puede vender más de lo que hay; por eso es el mínimo entre inventario y demanda.
- **Pedido (L):** implementa la política: se pide solo si el inventario final es estrictamente menor que s.

Verificación: la celda B318 debe mostrar 31/12/2026, A318 debe ser 313 y la suma de la columna D debe ser 12. Escriba temporalmente las demandas 4, 7 y 5 en `G6:G8` con $s = 6$ y $Q = 10$, y compare con el ejemplo de la sección 2. Luego restaure la fórmula aleatoria.

3.4. Paso A4. Resumen de la réplica

A la derecha de la tabla, calcule los resultados del año simulado:

Celda	Resultado	Fórmula
U6	Ganancia anual	`=SUMA(R6:R318)`
U7	Fracción de días con faltante	`=SUMA(K6:K318)/CONTAR(A6:A318)`
U8	Ventas perdidas promedio por día	`=PROMEDIO(I6:I318)`

Cada vez que presione F9, Excel genera nuevos números aleatorios y estas tres celdas cambian: cada recálculo es una réplica distinta del año.

3.5. Paso A5. Cien réplicas con una tabla de datos

1. En `W7:W106` escriba los números 1 a 100 (réplica).
2. En la fila 6 enlace los resultados: `X6` `=U6`, `Y6` `=U7`, `Z6` `=U8`. Deje `W6` vacía.
3. Seleccione el rango `W6:Z106`.
4. Vaya a Datos > Análisis de hipótesis > Tabla de datos.
5. Deje vacío "Celda de entrada (fila)". En "Celda de entrada (columna)" seleccione una celda vacía que no use ninguna fórmula, por ejemplo `W2`.
6. Acepte. Excel llena `X7:Z106` con 100 réplicas.

Por qué funciona: para cada fila, Excel escribe el número de réplica en la celda de entrada y recalcula la hoja. Como ninguna fórmula depende de esa celda, lo único que cambia son los números aleatorios, así que cada fila es un año simulado independiente.

Recomendaciones:

- Si el libro se vuelve lento, use Archivo > Opciones > Fórmulas > "Automático excepto en las tablas de datos" y recalcule con F9.

3.6. Paso A6. Estadísticos

Celda	Estadístico	Fórmula
AC5	Ganancia promedio	`=PROMEDIO(X7:X106)`
AC6	Desviación estándar de la ganancia	`=DESVEST.M(X7:X106)`
AC7	Probabilidad de ventas perdidas	`=PROMEDIO(Y7:Y106)`
AC8	Ventas perdidas promedio por día	`=PROMEDIO(Z7:Z106)`

`DESVEST.M` es la desviación estándar muestral (divide por $n-1$). Como todas las réplicas tienen 313 días, promediar las fracciones por réplica equivale a calcular la fracción sobre todos los días simulados.

3.7. Paso A7. Histograma de la ganancia (10 clases de igual ancho)

1. Calcule el mínimo (`AC12`) `=MIN(X7:X106)`, el máximo (`AC13`) `=MAX(X7:X106)` y el ancho de clase (`AC14`) `=(AC13-AC12)/10`.
2. Construya la tabla de clases en `AB16:AF26`: número de clase k (1 a 10), límite inferior, límite superior, marca de clase y frecuencia.

Columna	Contenido	Fórmula en la fila 17 (clase 1)
AB	Clase k	`1` (hasta 10)
AC	Límite inferior	`=$AC$12+(AB17-1)*$AC$14`
AD	Límite superior	`=AC17+$AC$14`
AE	Marca de clase (millones)	`=(AC17+AD17)/2/1000000`
AF	Frecuencia	(ver abajo)

Frecuencia de la clase 1 (fila 17), que se copia hasta la fila 26:

```
=CONTAR.SI.CONJUNTO($X$7:$X$106;">="&AC17;$X$7:$X$106;"<"&AD17)
```

En la última clase (fila 26) cambie `"<"` por `"<="` para incluir el máximo. La suma de las frecuencias debe ser 100.

3. Inserte un gráfico de columnas con las frecuencias (`AF17:AF26`) y las marcas de clase como etiquetas del eje horizontal. En el formato de la serie, ponga el ancho del intervalo en 0 % para que las barras queden juntas. Ponga título y nombre a los ejes.

3.8. Paso A8. Hoja de demanda empírica

Duplique la hoja `Uniforme` (clic derecho > Mover o copiar > Crear una copia) y llámela `Empirica`. Solo cambia la columna G:

```
=BUSCARV(ALEATORIO();TablaEmpirica;2;VERDADERO)
```

`ALEATORIO()` genera u uniforme en $[0;\ 1)$. `BUSCARV` con coincidencia aproximada (`VERDADERO`) busca el mayor límite inferior que no supera u y devuelve la demanda correspondiente: es la transformada inversa. Compruebe que la tabla de datos de la copia sigue funcionando; si no, bórrela y repita el paso A5.

3.9. Paso A9. Comparación de políticas

Cambie s y Q en la hoja `Parametros`, recalcule y registre los cuatro estadísticos de cada caso de demanda. Pruebe al menos 6 combinaciones (s, Q) distintas. Guarde los resultados (como valores) en una hoja `Politicas` con una fila por combinación y caso de demanda.

4. Parte B: modelo en Python (50 %)

El programa reproduce el mismo modelo de la Parte A en un solo archivo `.py`, construido por bloques. Se usan bucles explícitos para que cada línea corresponda a una columna de la hoja de Excel. Los comentarios del código están en inglés, como es estándar en la industria.

4.1. Paso B1. Importaciones y parámetros

Todos los parámetros se definen una sola vez al inicio. La semilla fija hace que sus resultados se puedan reproducir; use su número de documento o código de estudiante.

```python
import datetime as dt

import matplotlib.pyplot as plt
import numpy as np

# ---------------------------------------------------------------------------
# Parameters
# ---------------------------------------------------------------------------
YEAR = 2026
N_REPLICATIONS = 100
REORDER_POINT = 6          # s: order when final inventory is strictly below s <- your policy
ORDER_QTY = 10             # Q: constant order size (also the initial inventory) <- your policy

PRICE = 680_000            # revenue per unit sold
UNIT_COST = 600_000        # purchase cost per unit
ORDER_COST = 40_000        # fixed cost per order received
HOLDING_COST = 80          # per unit per day, applied to average daily inventory
FIXED_MONTHLY = 1_000_000 # charged on the first working day of each month

SEED = 12345               # use your student ID so your results are reproducible

# Empirical demand distribution (values and probabilities)
EMP_VALUES = [3, 4, 5, 6, 7]
EMP_PROBS = [0.20, 0.05, 0.40, 0.25, 0.10]
```

4.2. Paso B2. Calendario de días hábiles

La función recorre todas las fechas del año y descarta los domingos. Es el equivalente a la columna B de Excel.

```python
def build_calendar(year):
    """Return the list of working days (Monday-Saturday) of the given year.

    A plain loop over dates is used instead of pandas date ranges because it
    mirrors the Excel model (one row per working day).
    """
    days = []
    day = dt.date(year, 1, 1)
    while day.year == year:
        if day.weekday() != 6:  # weekday() == 6 is Sunday
            days.append(day)
        day += dt.timedelta(days=1)
    return days
```

Verifique que `len(build_calendar(2026))` sea 313.

4.3. Paso B3. Generadores de demanda

La demanda uniforme usa `integers`, que excluye el límite superior (por eso 8). La empírica implementa la transformada inversa con un recorrido de la probabilidad acumulada, igual que `BUSCARV` en Excel.

```python
def demand_uniform(rng):
    """Discrete uniform demand on {3, 4, 5, 6, 7}."""
    # integers() excludes the upper bound, hence 8
    return int(rng.integers(3, 8))


def demand_empirical(rng):
    """Discrete empirical demand generated by the inverse-transform method.

    The cumulative search is written explicitly (instead of rng.choice) so it
    matches the VLOOKUP-on-cumulative-probabilities logic used in Excel.
    """
    u = rng.random()
    cumulative = 0.0
    for value, prob in zip(EMP_VALUES, EMP_PROBS):
        cumulative += prob
        if u < cumulative:
            return value
    return EMP_VALUES[-1]  # guard against floating-point round-off
```

4.4. Paso B4. Simulación de un año (una réplica)

Esta función es el corazón del modelo. Recibe las demandas ya generadas, de modo que la lógica del inventario es determinística y se puede probar con demandas fijas.

```python
def simulate_year(demands, calendar, s, q, keep_log=False):
    """Simulate one year given a pre-generated demand sequence.

    Separating demand generation from the inventory logic makes the logic
    deterministic and testable: the same demand vector can be fed to this
    function and to the Excel model to check that both give the same profit.
    """
    profit = 0.0
    days_with_shortage = 0
    total_lost = 0
    log = []

    final_inv = 0
    order_pending = False

    for t, day in enumerate(calendar):
        # --- Receptions: day 1 receives the initial Q; later days receive
        #     the order placed at the end of the previous working day.
        received = q if (t == 0 or order_pending) else 0
        initial_inv = final_inv + received

        # --- Demand, sales and lost sales
        demand = demands[t]
        sales = min(initial_inv, demand)
```

```
        lost = demand - sales
        final_inv = initial_inv - sales

        # --- End-of-day review: order if final inventory is below s
        order_pending = final_inv < s

        # --- Daily cash flows
        month_start = (t == 0) or (calendar[t - 1].month != day.month)
        revenue = PRICE * sales
        purchase_cost = UNIT_COST * received
        ordering_cost = ORDER_COST if received > 0 else 0
        holding_cost = HOLDING_COST * (initial_inv + final_inv) / 2
        fixed_cost = FIXED_MONTHLY if month_start else 0
        daily_profit = (revenue - purchase_cost - ordering_cost
                        - holding_cost - fixed_cost)

        profit += daily_profit
        total_lost += lost
        days_with_shortage += 1 if lost > 0 else 0

        if keep_log:
            log.append((day, initial_inv, demand, sales, lost, final_inv,
                        int(order_pending), daily_profit))

    n_days = len(calendar)
    return {
        "profit": profit,
        "shortage_fraction": days_with_shortage / n_days,
        "lost_per_day": total_lost / n_days,
        "log": log,
    }
```

Prueba obligatoria con el ejemplo de la sección 2. Las utilidades diarias deben ser $-4.320.640$, $4.079.760$ y $-2.640.600$:

```
cal = build_calendar(YEAR)[:3]
res = simulate_year([4, 7, 5], cal, s=6, q=10, keep_log=True)
for row in res["log"]:
    print(row)
```

4.5. Paso B5. Cien réplicas para una distribución

Cada réplica genera 313 demandas nuevas y simula el año. Se guardan los resultados de cada réplica y luego se calculan los estadísticos.

```
def run_experiment(demand_fn, rng, calendar, s, q, n_reps):
    profits = np.zeros(n_reps)
    shortage_frac = np.zeros(n_reps)
    lost_per_day = np.zeros(n_reps)

    for r in range(n_reps):
        demands = [demand_fn(rng) for _ in calendar]
        res = simulate_year(demands, calendar, s, q)
        profits[r] = res["profit"]
        shortage_frac[r] = res["shortage_fraction"]
```

```
        lost_per_day[r] = res["lost_per_day"]

    # All replications have the same number of days, so averaging the
    # per-replication fractions equals pooling all simulated days.
    return {
        "profits": profits,
        "mean_profit": profits.mean(),
        "std_profit": profits.std(ddof=1),  # sample std, like Excel STDEV.S
        "prob_lost_sales": shortage_frac.mean(),
        "mean_lost_per_day": lost_per_day.mean(),
    }
```

`ddof=1` hace que la desviación estándar sea muestral (divide por $n-1$), igual que `DESVEST.M`.

4.6. Paso B6. Histograma

`bins=10` crea 10 clases de igual ancho entre el mínimo y el máximo, igual que en Excel.

```
def plot_histogram(profits, title, filename):
    """Histogram with 10 equal-width classes between min and max profit."""
    fig, ax = plt.subplots(figsize=(8, 4.5))
    ax.hist(profits / 1e6, bins=10, edgecolor="black")
    ax.set_title(title)
    ax.set_xlabel("Annual profit (millions COP)")
    ax.set_ylabel("Frequency")
    fig.tight_layout()
    fig.savefig(filename, dpi=150)
    plt.close(fig)
```

4.7. Paso B7. Comparación de políticas

Para comparar políticas en igualdad de condiciones, el generador se reinicia con la misma semilla para cada política (números aleatorios comunes). Así todas enfrentan las mismas demandas y la diferencia entre ellas se debe a la política, no al azar.

```
def compare_policies(policies, demand_fn, calendar, n_reps, seed):
    """Evaluate several (s, Q) policies and return one summary row per policy.

    The generator is re-created with the same seed for every policy (common
    random numbers): all policies face the same demand sequences, so the
    differences between them come from the policy, not from sampling noise.
    """
    rows = []
    for s, q in policies:
        rng = np.random.default_rng(seed)
        res = run_experiment(demand_fn, rng, calendar, s, q, n_reps)
        rows.append((s, q, res["mean_profit"], res["std_profit"],
                     res["prob_lost_sales"], res["mean_lost_per_day"]))
    return rows
```

4.8. Paso B8. Programa principal

Ejecuta la política elegida para ambos casos de demanda, guarda los histogramas como imágenes PNG y luego compara una lista de políticas. Reemplace la lista `policies` por sus propias combinaciones (al menos 20).

```python
def main():
    rng = np.random.default_rng(SEED)
    calendar = build_calendar(YEAR)
    print(f"Working days in {YEAR}: {len(calendar)}")
    print(f"Policy: s = {REORDER_POINT}, Q = {ORDER_QTY}, "
          f"replications = {N_REPLICATIONS}\n")

    cases = [("Uniform", demand_uniform), ("Empirical", demand_empirical)]
    for name, fn in cases:
        res = run_experiment(fn, rng, calendar, REORDER_POINT, ORDER_QTY,
                             N_REPLICATIONS)
        print(f"--- {name} demand ---")
        print(f"Mean annual profit : {res['mean_profit']:,.0f}")
        print(f"Std annual profit  : {res['std_profit']:,.0f}")
        print(f"P(lost sales/day)  : {res['prob_lost_sales']:.4f}")
        print(f"Mean lost units/day: {res['mean_lost_per_day']:.4f}\n")
        plot_histogram(res["profits"], f"Annual profit - {name} demand",
                       f"hist_{name.lower()}.png")

    # Policy comparison (example grid; students choose their own candidates)
    policies = [(s, q) for s in (5, 6, 7) for q in (15, 30, 60, 100)]
    for name, fn in cases:
        print(f"--- Policy comparison, {name} demand ---")
        print(f"{'s':>3} {'Q':>4} {'mean profit':>14} {'std':>12} "
              f"{'P(lost)':>8} {'lost/day':>9}")
        rows = compare_policies(policies, fn, calendar, N_REPLICATIONS, SEED)
        for s, q, mean, std, p_lost, lost in sorted(rows, key=lambda r: -r[2]):
            print(f"{s:>3} {q:>4} {mean:>14,.0f} {std:>12,.0f} "
                  f"{p_lost:>8.4f} {lost:>9.4f}")
        print()


if __name__ == "__main__":
    main()
```

Sus resultados no serán idénticos a los de Excel porque cada herramienta usa números aleatorios distintos. Sí deben ser comparables: del mismo orden de magnitud y con diferencias explicables por el azar.

5. Análisis

Cada parte se entrega con su propio documento de análisis. Responda con base en sus resultados y justifique cada afirmación con números, tablas o gráficos.

Análisis de la Parte A (Excel)

1. Para la política (s, Q) que recomienda, reporte la ganancia promedio, la desviación estándar de la ganancia, la probabilidad de ventas perdidas y las ventas perdidas promedio por día, para ambos casos de demanda. Incluya los dos histogramas. Justifique.
2. Calcule la media y la varianza teóricas de la demanda diaria en cada caso. ¿En qué se parecen las dos distribuciones y en qué se diferencian? ¿Cómo se refleja esa diferencia en las ventas perdidas y en la ganancia?
3. Presente la tabla de la hoja `Politicas` (al menos 6 combinaciones). Elija la mejor política con un criterio explícito y justifíquelo: ganancia promedio, riesgo (desviación estándar) y nivel de servicio (ventas perdidas).
4. Describa la forma de los histogramas. ¿Qué explica la variabilidad de la ganancia anual entre réplicas? *Pista:* revise el inventario final del 31/12/2026 y el supuesto 10.
5. Demuestre que, con estos supuestos, si $s \geq 7$ y $Q \geq 7$ nunca hay ventas perdidas. ¿Lo confirman sus resultados?
6. Con s fijo, ¿cómo cambia la ganancia promedio al aumentar Q? Explique qué costos pesan más en esa tendencia.

Análisis de la Parte B (Python)

1. Reporte los mismos estadísticos e histogramas para la política que eligió en Excel. Compárelos con los de Excel en una tabla. ¿Son iguales? ¿Deberían serlo? Explique.
2. Ejecute su política con 5 semillas distintas. ¿Cuánto varía la ganancia promedio entre semillas? ¿Qué le dice eso sobre la precisión de un estimado basado en 100 réplicas?
3. Use `compare_policies` con una rejilla más amplia que la de Excel (al menos 20 combinaciones). ¿La mejor política coincide con la que eligió en Excel? Si no coincide, explique por qué.
4. Explique por qué `compare_policies` usa la misma semilla para todas las políticas. ¿Qué pasaría con la comparación si cada política usara números aleatorios distintos?
5. Compare Excel y Python para este modelo: tiempo de cómputo, facilidad para cambiar el número de réplicas o de políticas, trazabilidad de los cálculos y riesgo de errores.

6. Entregables y rúbrica

El taller es individual. Se deben cargar todos los archivos en EAFIT Interactiva; **no se admiten links**. Entregue tres archivos con el formato `Apellido_Nombre_TallerN`:

Archivo	Contenido
`Apellido_Nombre_TallerN.xlsx`	Hojas `Parametros`, `Uniforme`, `Empirica` y `Politicas`, con la política recomendada cargada
`Apellido_Nombre_TallerN.py`	Programa completo; debe ejecutarse sin errores con `python Apellido_Nombre_TallerN.py`
`Apellido_Nombre_TallerN_Python.pdf`	Análisis de los experimentos

Parte A: Excel (50 %)

Criterio	Puntos
Modelo diario correcto: calendario, política, inventarios, ingresos y costos; reproduce el ejemplo de verificación	15
Tabla de datos con 100 réplicas y estadísticos correctos	10
Histograma con 10 clases de igual ancho, frecuencias que suman 100 y gráfico rotulado	5
Comparación de al menos 6 políticas (hoja `Politicas`)	5
Análisis: preguntas 1 a 6 de la Parte A	15
Total Parte A	**50**

Parte B: Python (50 %)

Criterio	Puntos
Código correcto que reproduce el ejemplo de verificación	15
Estadísticos e histogramas para ambos casos de demanda	10
Comparación de al menos 20 políticas con números aleatorios comunes	5
Calidad del código: estructura en funciones, comentarios, semilla fija	5
Análisis: preguntas 1 a 5 de la Parte B	15
Total Parte B	**50**